

RELAZIONE TECNICA DI PROGETTO

Corso: Web and Multimedia Technologies (Laurea Magistrale) **Docente:** Prof. Marco Porta — Università degli Studi di Pavia **Studente:** Marco Santoro **Progetto:** Algora Studio — sito istituzionale e piattaforma di contatto **Anno Accademico:** 2025/2026

1. INTRODUZIONE E CONTESTO APPLICATIVO

Questa relazione descrive la progettazione e le scelte tecniche del sito web di **Algora Studio**, uno studio di sviluppo software specializzato in soluzioni verticali per il patrimonio culturale e la memoria storica documentale italiana (Archivi di Stato, archivi storici comunali, studi notarili, fondazioni).

Il prodotto attorno a cui ruota la comunicazione del sito è **Foliarium**, un gestionale sviluppato per l'Archivio di Stato di Savona per la digitalizzazione, la modellazione relazionale e la consultazione *fuzzy* degli archivi catastali storici (dal 1830 a oggi).

Motivazione della scelta

La maggior parte dei siti per software house si appoggia a template generici o a CMS che frammentano il controllo sul codice. Realizzare un portale ad hoc risponde all'obiettivo di dimostrare che la programmazione web nativa (**HTML5, CSS3, JavaScript Vanilla, PHP e MySQL/MariaDB**) permette di ottenere:

1. un'estetica caratterizzata ("Archivio Caldo") coerente con il dominio trattato;
 2. un carico di rete ridotto, con un solo foglio di stile e un solo file JavaScript;
 3. conformità alla validazione W3C e alle regole di base di usabilità e accessibilità viste a lezione.
-

2. ARCHITETTURA DELL'INFORMAZIONE E MAPPATURA DEI FILE

Il sito si articola su **6 pagine di contenuto**, servite da PHP, più **1 script server-side** per la persistenza dei dati e **1 schermata di servizio** riservata a chi gestisce il sito. Le parti comuni sono estratte in due **include PHP** riusati da tutte le pagine.

File	Ruolo
index.php	Home. Vision dello studio, manifesto, metriche di sintesi, anteprima del caso studio, sezione approccio, call to action finale.
chi-siamo.php	Profilo dello studio. Filosofia dello sviluppo verticale, i tre valori guida, profilo del fondatore, stack tecnologico, prospettive future.
foliarium.php	Scheda prodotto. Funzionalità di Foliarium (ricerca fuzzy, albero delle proprietà, audit trail, esportazione report), requisiti di sistema, licenze e pacchetti di assistenza.

File	Ruolo
caso-studio.php	Caso studio Archivio di Stato di Savona. Risultati quantitativi (69 comuni, 12.000+ partite, 8.500+ possessori), confronto “prima e dopo”, fasi del progetto.
contatti.php	Contatti e demo. Guida in quattro fasi, modulo di contatto con consenso al trattamento, FAQ a fisarmonica.
privacy.php	Informativa privacy. Dati raccolti, finalità, base giuridica, conservazione, diritti dell’interessato, sezione sui cookie.
nav.php	Include: barra di navigazione, generata da un array PHP che marca automaticamente la voce attiva.
footer.php	Include: piè di pagina comune a tutte le pagine.
invia-contatto.php	Backend. Riceve il POST del modulo, valida, inserisce nel database con istruzione preparata, restituisce la pagina di esito.
archivio.php	Area riservata. Schermata di servizio protetta da password: elenco delle richieste, inserimento manuale, modifica ed eliminazione (§5.7).
config-db.php	Parametri di connessione al database, isolati dalla logica applicativa.
config-admin.php	Impronta della password dell’area riservata, isolata come i parametri del database.
crea_db.sql	Script DDL di creazione di database e tabella.
style.css	Foglio di stile unico dell’intero sito (1.503 righe, ~55 KB).
main.js	Comportamenti client-side (63 righe).
fonts/	I due caratteri tipografici in formato WOFF2 (8 file, 300 KB).
img/	Cartella per le schermate del prodotto, con le istruzioni in LEGGIMI.md.

2.1 Perché PHP anche sulle pagine di contenuto

Le cinque pagine non contengono logica applicativa, ma hanno estensione .php per poter usare include:

```
<?php $active = 'home'; ?>
...
<?php include 'nav.php'; ?>
```

Barra di navigazione e piè di pagina esistono così in **un’unica copia**. La voce di menu corrispondente alla pagina corrente viene evidenziata confrontando la variabile

\$active con le chiavi dell'array che descrive il menu:

```
$navItems = [  
    'home'      => ['href' => 'index.php',    'label' => 'Home'],  
    'chi-siamo' => ['href' => 'chi-siamo.php', 'label' => 'Chi siamo'],  
    /* ... */  
];  
foreach ($navItems as $key => $item) {  
    /* class="active" quando $active === $key */  
}
```

Senza include, una modifica al menu andrebbe replicata a mano su sei file, con il rischio concreto di disallineamenti.

3. FRONTEND: HTML5 SEMANTICO, CSS3 E DESIGN SYSTEM

3.1 Scrittura nativa e struttura semantica

Il codice è scritto interamente a mano, senza editor WYSIWYG e senza framework CSS (Bootstrap, Tailwind). Ogni pagina usa i tag strutturali di HTML5: <nav> per la navigazione, <main> per il contenuto principale, <header> per le sezioni di apertura, <section> per i blocchi tematici, <footer> per la chiusura.

L'intero contenuto di ogni pagina è racchiuso in <main id="contenuto">: questo definisce il *landmark* principale e permette il funzionamento del collegamento di salto descritto in §6.

3.2 Design system “Archivio Caldo”

La palette e la tipografia sono definite con **variabili CSS** (custom properties) raccolte in :root, così che un cambio di tonalità si propaghi a tutto il sito modificando una riga sola:

- **Colori:** fondo pergamena (--parchment: #F4EFE4, --cream: #F9F6EF), testo e contenitori inchiostro (--ink: #1E150A), accenti dorati (--gold: #B8821A), verde archivio per le conferme (--green: #1A3A28).
- **Tipografia:** *Cormorant Garamond* per i titoli display e *Libre Baskerville* per il testo corrente, entrambi da Google Fonts; *Trebuchet MS / Calibri* in maiuscolo spaziato per etichette e micro-testi.

I due caratteri erano inizialmente caricati con un @import da fonts.googleapis.com. Era l'**unica richiesta a un dominio terzo** dell'intero sito, e non è un dettaglio tecnico: ogni visita comunicava l'indirizzo IP dell'utente a un server esterno, senza che ne fosse informato e senza che avesse modo di opporsi.

I file WOFF2 sono quindi stati **scaricati e ospitati sul sito stesso**, in fonts/, e l'@import è stato sostituito da otto dichiarazioni @font-face:

```
@font-face {  
    font-family: 'Cormorant Garamond';  
    font-style: normal;  
    font-weight: 400;  
    font-display: swap;  
    src: url('fonts/cormorant-garamond-400.woff2') format('woff2');
```

```
    unicode-range: U+0000-00FF, U+0131, U+0152-0153, /* ... */;  
}
```

Sono inclusi soltanto gli otto tagli effettivamente usati dal foglio di stile e soltanto il subset latin, che copre l'italiano accentato e la punteggiatura tipografica: in tutto 300 KB. I quattro segni presenti nelle pagine che escono da quel subset (→ ● ○ ★) ricadono sul carattere di sistema, esattamente come accadeva prima, perché nessun subset servito da Google li conteneva.

La direttiva `font-display: swap` mostra subito il testo con il carattere di ripiego e lo sostituisce a caricamento concluso, evitando l'intervallo di pagina vuota che si avrebbe con il comportamento predefinito.

Il sito non effettua oggi **nessuna richiesta a domini esterni**: è la premessa che rende possibile quanto descritto in §6.6.

Come descritto in §6.2, l'oro di marca è usato per filetti, bordi e sfondi, mentre **come colore di testo** il foglio di stile ricorre a due varianti calibrate sul fondo, perché la tonalità originale non raggiungeva il contrasto minimo richiesto.

Al design system appartengono anche i **nomi delle classi**. Nella prima stesura ogni pagina aveva battezzato a modo suo componenti identici: la stessa griglia di card compariva con undici nomi diversi (`.ph-grid`, `.val-grid`, `.feat-grid`, `.lic-grid`, `.ris-grid`...), l'occhiello sopra il titolo con altrettanti (`.ph-num`, `.vn`, `.feat-n`, `.ris-n`...), e la card in evidenza era marcata di volta in volta `.active`, `.dark`, `.featured`, `.after` o — su *chi siamo* — da un `:nth-child(2)` che dipendeva dalla posizione nella griglia. Il foglio pagava quella traduzione a ogni regola, sotto forma di elenchi di selettori lunghi cinque o sei voci.

I componenti che si ripetono hanno ora un nome solo: `.card-grid` (con `.card-grid-2` per le griglie a due colonne), `.label-sm` per l'occhiello — la stessa idea di `.label`, un gradino più in piccolo — `.card-dark` per la variante scura, `.card-price` per la cifra in caratteri display, `.note-box` per il riquadro su fondo oro tenue. Sono rimaste le classi che dicono *cosa* è un elemento e non solo come è colorato (`.active` per la fase in corso, `.before` e `.after` per il confronto prima e dopo), mentre quelle puramente descrittive (`.dark`, `.featured`) sono sparite. Restano al loro posto anche i nomi delle sezioni (`.valori`, `.problema`, `.faq`...): raggruppano più sezioni sotto uno stesso sfondo, ma descrivono il contenuto, e sostituirli con classi come `.sfondo-crema` sposterebbe nell'HTML una decisione che appartiene al foglio di stile.

Il consolidamento ha reso visibile una dipendenza dalla specificità che i nomi lunghi tenevano nascosta. `.ph-card.active` (due classi) vinceva su `.ph-card` (una) a prescindere dalla posizione nel foglio; `.card-dark` da sola perderebbe invece contro il fondo che ogni pagina assegna alla propria card, dichiarato più avanti. Le due regole che dipingono la card scura sono perciò scritte a partire dalla griglia che la contiene, `.card-grid .card-dark`, che è esattamente il gradino di specificità mancante. Lo stesso vale per l'occhiello: essendo un `<p>`, perde il confronto con la regola che colora tutti i paragrafi della card, e dove quella regola esiste il colore va dichiarato un livello più in profondità.

3.3 Layout non lineare

Il requisito di un layout non semplicemente lineare è soddisfatto da tre tecniche, tutte fuori dal flusso normale del documento:

1. **Barra di navigazione ancorata:** #nav usa `position: fixed; top: 0; left: 0; right: 0; z-index: 200`, restando visibile durante lo scorrimento.
2. **Griglie asimmetriche con CSS Grid:** la hero e il caso studio usano colonne di larghezza diversa (`grid-template-columns: 1fr 400px`) e griglie a `gap: 1px` su fondo colorato, che producono l'effetto di "caselle" archivistiche separate da un filetto. In tutto il foglio di stile sono presenti 18 contesti di griglia.
3. **Confronto "prima e dopo":** in `caso-studio.php`, due colonne a contrasto invertito (pergamena contro inchiostro) affiancano il processo manuale e quello digitalizzato.

A questi si aggiungono elementi in `position: absolute` usati come decorazione (la lettera "A" in filigrana nella hero, i punti della timeline).

3.4 Comportamento responsive: impostazione mobile-first

Il foglio di stile è scritto **mobile-first**: le regole di base descrivono lo schermo stretto, e un unico punto di rottura a 961px introduce la disposizione desktop tramite `@media` (`min-width: 961px`).

Concretamente, ogni griglia nasce a colonna singola:

```
.feat-grid {
  display: grid; grid-template-columns: 1fr;
  gap: 24px; background: var(--border);
}
```

e le colonne multiple arrivano dal blocco desktop della sezione:

```
@media (min-width: 961px) {
  .feat-grid { grid-template-columns: 1fr 1fr; gap: 2px; }
}
```

I blocchi `@media` non sono raccolti in coda al file ma collocati **al termine della sezione che riguardano** — uno per le griglie comuni, uno per il piè di pagina, uno per ciascuna pagina — così che la variante desktop di un componente si legga accanto alla sua definizione di base.

Il passo orizzontale delle fasce a tutta larghezza è centralizzato in una variabile, ridichiarata una sola volta per il desktop:

```
:root { --pad-x: 20px; }
@media (min-width: 961px) {
  :root { --pad-x: clamp(24px, 5vw, 72px); }
}
```

Undici regole (barra di navigazione, sezioni, fasce, piè di pagina, intestazioni di pagina) usano `var(--pad-x)`: il margine di pagina si modifica in un punto solo.

Perché non uno strato correttivo `max-width` L'impostazione precedente definiva il layout desktop nelle regole di base e lo correggeva a valle con un blocco `@media` (`max-width: 960px`) che riportava a colonna singola tutte le griglie. Quel blocco aveva bisogno di **nove dichiarazioni !important** per vincere sulle regole che stava annullando, ed è un difetto strutturale, non estetico: !important scavalca la specificità, quindi una regola mirata perde contro una regola generica dichiarata dopo di essa.

Il caso concreto emerso in questo progetto: i titoli di colonna del piè di pagina sono governati da `.footer-col h2 { font-size: 10px }`, ma lo strato correttivo conteneva

h2 { font-size: clamp(28px, 6vw, 36px) !important }. Su schermo stretto vinceva quest'ultima, e le etichette "Studio", "Prodotti", "Contatti" venivano rese a 28px anziché a 10px — visibile solo sotto i 960px. Rimosso lo strato, la regola specifica torna a valere e il difetto sparisce senza alcun intervento mirato.

Nel foglio di stile non resta oggi nessun !important di layout: gli unici tre superstiti sono quelli, idiomatici, del blocco prefers-reduced-motion, dove servono proprio a scavalcare qualunque animazione dichiarata altrove.

4. CLIENT-SIDE: JAVASCRIPT VANILLA

main.js contiene tre comportamenti, scritti in JavaScript ES6+ senza librerie.

- **Menu di navigazione (mobile).** Il pulsante apre e chiude l'elenco dei collegamenti, aggiornando l'attributo aria-expanded; il tasto Esc chiude il menu e riporta il focus sul pulsante che lo aveva aperto.
- **Ombra della barra allo scorrimento.** Un listener su scroll aggiunge la classe .scrolled a #nav oltre i 20 px, staccando visivamente la barra dal contenuto.
- **FAQ a fisarmonica.** Al clic su una domanda le altre risposte si chiudono; l'apertura è animata sulla proprietà max-height e comunicata alle tecnologie assistive tramite aria-expanded.

Entrambi i comandi interattivi sono elementi **<button> nativi**: la scelta è discussa in §6.1.

5. SERVER-SIDE E PERSISTENZA DATI

5.1 Architettura

Un modulo che invii una email via mailto: o simuli l'invio in JavaScript non garantisce né persistenza né tracciabilità. Il progetto adotta quindi l'architettura a tre livelli **Client (HTML/CSS/JS) → Application server (PHP 8) → Database server (MySQL/MariaDB)**.

5.2 Base di dati (algora_db)

```
CREATE DATABASE IF NOT EXISTS algora_db
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE algora_db;

CREATE TABLE IF NOT EXISTS contatti (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  nome        VARCHAR(100) NOT NULL,
  ente        VARCHAR(100),
  email       VARCHAR(100) NOT NULL,
  tipo        VARCHAR(50),
  messaggio   TEXT          NOT NULL,
  consenso_privacy TINYINT(1) NOT NULL DEFAULT 0,
  data_consenso DATETIME     DEFAULT NULL,
  data_invio   DATETIME     DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

La codifica utf8mb4 copre l'intero repertorio Unicode, inclusi i caratteri accentati e i segni tipografici usati nei nomi di enti e località.

5.3 Flusso di elaborazione

1. **Verifica del metodo HTTP.** Una richiesta che non sia POST (tipicamente l'apertura diretta dell'URL) non ha dati da elaborare: lo script risponde con un redirect 303 See Other verso contatti.php, invece di mostrare una pagina di errore priva di significato.
2. **Validazione.** trim() sui campi, controllo dei campi obbligatori, filter_var(\$email, FILTER_VALIDATE_EMAIL) per la correttezza formale dell'indirizzo. Messaggi di errore distinti per campi mancanti e per email non valida.
3. **Inserimento con istruzione preparata.**

```
$stmt = $conn->prepare(
    'INSERT INTO contatti (nome, ente, email, tipo, messaggio)
    VALUES (?, ?, ?, ?, ?)'
);
$stmt->bind_param('sssss', $nome, $ente, $email, $tipo, $messaggio);
$stmt->execute();
```

I segnaposto ? e il binding separato dei parametri tengono distinti comandi SQL e dati forniti dall'utente: il contenuto dei campi non può essere reinterpretato come istruzione. Un invio con nome = "Rossi'); DROP TABLE contatti; --" viene memorizzato come stringa letterale e la tabella resta intatta.

4. **Pagina di esito.** In base all'esito lo script produce una pagina di conferma personalizzata o una scheda di errore, entrambe con la navigazione, il piè di pagina e il foglio di stile del resto del sito.

5.4 Escaping: una sola volta, in uscita

I dati vengono salvati nel database **così come l'utente li ha scritti**, e htmlspecialchars() viene applicato soltanto al momento di stamparli nella pagina.

Applicare l'escaping anche in ingresso — errore frequente, perché sembra “più sicuro” — produce una doppia codifica: un cognome come Sant'Angelo verrebbe memorizzato come Sant'Angelo e ristampato a video come Sant&#039;Angelo. Il database si riempirebbe di entità HTML al posto dei caratteri reali, rendendo inutilizzabili i dati per qualsiasi uso diverso dalla pagina web (esportazioni, email, ricerche). La protezione dalle *SQL injection* è affidata all'istruzione preparata, non all'escaping HTML, che serve a un problema diverso (*Cross-Site Scripting*, in uscita).

5.5 Gestione degli errori e credenziali

I parametri di connessione stanno in config-db.php, separati dalla logica: il passaggio da ambiente locale a hosting richiede di modificare un solo file, e in un deployment reale il file va escluso dal versionamento.

A partire da PHP 8.1 l'estensione mysqli segnala gli errori **sollevando eccezioni** anziché valorizzando connect_error. Un controllo scritto nella forma tradizionale if (\$conn->connect_error) non verrebbe quindi mai eseguito, e un database irraggiungibile produrrebbe una traccia di stack contenente host, utente e percorso del file. La connessione è perciò racchiusa in un try/catch: all'utente arriva un messaggio generico, il dettaglio tecnico finisce nel log del server tramite error_log().

5.6 Consenso al trattamento

Il modulo raccoglie dati personali e li scrive in un database: la base giuridica su cui poggia il trattamento è il consenso dell'interessato. Prima del pulsante di invio è quindi presente una casella obbligatoria che rimanda all'informativa:

```
<div class="form-consenso">
  <input type="checkbox" id="consenso" name="consenso" value="1" required>
  <label for="consenso">Ho letto l'<a href="privacy.php">informativa
    privacy</a> e acconsento al trattamento dei miei dati personali
    per ricevere una risposta a questa richiesta.</label>
</div>
```

L'attributo `required` fa segnalare l'omissione dal browser, ma il controllo che conta è quello lato server, perché la validazione client si aggira banalmente:

```
$consenso = isset($_POST['consenso']) && $_POST['consenso'] === '1';
```

Senza consenso lo script non esegue alcun inserimento e restituisce un messaggio di errore specifico. Quando invece il consenso c'è, non viene registrato soltanto il fatto che la casella fosse spuntata: l'articolo 7 del Regolamento richiede di poter **dimostrare** che il consenso è stato prestato, quindi la tabella conserva anche il momento in cui è avvenuto, nelle colonne `consenso_privacy` e `data_consenso`.

5.7 Area riservata: le altre tre operazioni sull'archivio

Il flusso descritto fin qui esegue **una sola** delle quattro operazioni fondamentali su una tabella: l'inserimento. Lettura, modifica ed eliminazione restavano possibili soltanto da phpMyAdmin, cioè dal pannello di amministrazione del DBMS. È un limite pratico e insieme giuridico: la sezione 4 dell'informativa promette che le richieste vengono cancellate al termine del periodo di conservazione, e gli articoli 16 e 17 del Regolamento riconoscono all'interessato il diritto di ottenere rettifica e cancellazione dei propri dati. Una promessa che si può mantenere solo aprendo il pannello del database è una promessa fragile.

`archivio.php` completa il quadro con una schermata di servizio, protetta da password e raggiungibile da un collegamento discreto nel piè di pagina. Le quattro operazioni si distribuiscono così:

Operazione	Istruzione SQL	Come viene richiesta
Lettura (<i>Read</i>)	SELECT	GET <code>archivio.php</code>
Inserimento (<i>Create</i>)	INSERT	POST con azione=crea
Modifica (<i>Update</i>)	UPDATE	POST con azione=aggiorna
Eliminazione (<i>Delete</i>)	DELETE	POST con azione=elimina

Un solo file, tre viste Elenco, modifica e conferma di eliminazione sono rami della stessa pagina, non file distinti: la sequenza **richiesta** → **azione** → **risposta** resta leggibile dall'alto in basso, come in `invia-contatto.php`. In testa allo script stanno le azioni che scrivono, subito sotto le interrogazioni di lettura, e solo in fondo il markup. Nessuna riga di HTML viene prodotta prima che si sappia come è andata l'operazione: è la condizione perché il redirect del punto successivo possa funzionare, dato che le intestazioni HTTP vanno inviate prima del corpo della risposta.

Le scritture viaggiano solo in POST Un collegamento in GET viene seguito dai crawler dei motori di ricerca e ripetuto dal tasto “aggiorna” del browser: affidare a un GET la cancellazione di una riga significa consegnare l’archivio al primo indicizzatore che passa. Nella schermata i collegamenti “Modifica” ed “Elimina” sono effettivamente GET, ma non toccano nulla: aprono soltanto la vista corrispondente. Tutto ciò che scrive è un POST.

Post/Redirect/Get Dopo ogni scrittura lo script non produce direttamente una pagina, ma risponde con un redirect 303 See Other verso sé stesso. Ricaricare la pagina di esito ripete quindi una lettura, non l’inserimento o l’eliminazione appena eseguiti — il classico problema del modulo inviato due volte per un F5 di troppo. Il messaggio di esito e, in caso di errore di validazione, i valori digitati vengono trasportati **in sessione** e non nella *querystring*: un messaggio ripreso dall’URL sarebbe testo di provenienza esterna stampato in pagina, cioè esattamente ciò che si evita in §5.4.

Token anti-CSRF Il cookie di sessione viene allegato dal browser a **qualsiasi** richiesta diretta al sito, anche a quella partita da una pagina ostile aperta in un’altra scheda. Senza contromisure, un modulo nascosto su un sito di terzi potrebbe inviare a `archivio.php` un POST di eliminazione, e il server lo eseguirebbe come se fosse legittimo. La contromisura è un valore casuale noto soltanto alla sessione, ricopiato in ogni modulo della pagina e verificato a ogni scrittura:

```
if (empty($_SESSION['csrf'])) {
    $_SESSION['csrf'] = bin2hex(random_bytes(32));
}
/* ... a ogni operazione che scrive ... */
if (!hash_equals($_SESSION['csrf'], $_POST['csrf'] ?? '')) {
    /* richiesta respinta */
}
```

Una pagina di terzi non può leggere quel valore, quindi non può costruire una richiesta valida. Il confronto usa `hash_equals()` e non l’operatore `===` perché il primo non si ferma al primo carattere diverso: il tempo di risposta non lascia intuire quanto ci si è avvicinati al token.

Autenticazione Il file `config-admin.php` non contiene la password ma la sua **impronta** calcolata con `password_hash()`, isolata dalla logica applicativa come i parametri del database:

```
if (password_verify($_POST['password'] ?? '', $cfgAdmin['password_hash'])) {
    session_regenerate_id(true);
    $_SESSION['archivio_admin'] = true;
}
```

L’algoritmo `bcrypt` è lento per costruzione e applica un *sale* diverso a ogni chiamata: due impronte della stessa password sono diverse fra loro e non sono confrontabili con una tabella precalcolata. `session_regenerate_id(true)` sostituisce l’identificativo di sessione nel momento in cui cambiano i privilegi, così un identificativo eventualmente imposto prima dell’accesso non vale più nulla (*session fixation*). Il messaggio di errore è unico e generico: a un estraneo non si conferma mai quale metà della credenziale ha indovinato.

I limiti dell’impostazione sono dichiarati nel commento in testa al file: una password unica condivisa è sufficiente per un pannello dimostrativo su una singola postazione,

mentre un archivio in produzione richiederebbe utenze nominali, HTTPS obbligatorio, limitazione dei tentativi di accesso e registro degli accessi.

La conferma di eliminazione è una pagina, non una finestra di dialogo Un `confirm()` JavaScript sparirebbe con lo script disattivato, lasciando un pulsante che cancella al primo clic. La conferma è quindi una vista a sé, raggiunta in GET, che mostra per esteso il dato in procinto di sparire — nome, indirizzo, data e testo integrale del messaggio — e contiene il solo modulo POST che esegue davvero l’eliminazione. Funziona senza JavaScript e dà modo di accorgersi di aver scelto la riga sbagliata.

La data del consenso non si riscrive a ogni salvataggio Se la modifica sovrascivesse `data_consenso` con `NOW()` a ogni salvataggio, la prova richiesta dall’articolo 7 del Regolamento (§5.6) diventerebbe la data dell’ultima correzione di un refuso, non il momento in cui il consenso è stato prestato. La colonna viene perciò valorizzata solo alla prima spunta e azzerata in caso di revoca:

```
UPDATE contatti
  SET nome = ?, ente = ?, email = ?, tipo = ?, messaggio = ?,
      consenso_privacy = ?,
      data_consenso = IF(? = 1, COALESCE(data_consenso, NOW()), NULL)
WHERE id = ?
```

Continuità con il resto del progetto Ogni interrogazione passa da un’istruzione preparata e ogni dato stampato a video passa da `htmlspecialchars()`: le stesse due regole di §5.3 e §5.4, applicate qui a un numero molto maggiore di valori in uscita. La schermata riusa i componenti già definiti nel foglio di stile e, su schermo stretto, la tabella non si comprime ma scorre orizzontalmente dentro un contenitore dichiarato `role="region"` e focalizzabile da tastiera, che si può quindi scorrere anche senza mouse; il messaggio di esito è marcato `role="status"`, perché venga annunciato dai lettori di schermo senza sottrarre il focus. La pagina porta infine `<meta name="robots" content="noindex, nofollow">`: una schermata di servizio non ha ragione di comparire nei motori di ricerca.

6. USABILITÀ E ACCESSIBILITÀ

L’accessibilità è stata trattata come requisito di progetto e non come rifinitura finale. Le verifiche sono state condotte con il **W3C Nu Markup Validation Service** per la correttezza formale e con **axe-core** (motore di analisi automatica delle regole WCAG) per il comportamento, su tutte le pagine e su due larghezze di viewport (1440 px e 390 px).

6.1 Comandi interattivi azionabili da tastiera

Il menu a scomparsa e la fisarmonica delle FAQ erano inizialmente `<div>` con un gestore onclick. Un `<div>` non è raggiungibile con il tasto Tab e non risponde a Invio o Barra spaziatrice: entrambi i comandi funzionavano solo con il mouse. L’aggiunta di `role="button"` e `tabindex="0"` al solo elemento del menu peggiorava anzi la situazione, perché faceva annunciare l’elemento come pulsante a uno screen reader senza renderlo effettivamente azionabile.

Entrambi sono stati riscritti come elementi `<button type="button">` **nativi**. Un pulsante nativo è già nell'ordine di tabulazione, risponde a Invio e Barra spaziatrice senza codice aggiuntivo, viene annunciato correttamente e riceve gli stili di focus del sistema. Lo stato di apertura è esposto con `aria-expanded`, e `aria-controls` collega il comando al blocco che governa:

```
<button type="button" class="faq-trigger"
      aria-expanded="false" aria-controls="faq-a1">
  <span>Quanto tempo ci vuole per installare Foliarium?</span>
  <span class="faq-toggle" aria-hidden="true">+</span>
</button>
```

Il segno + che ruota in × è decorativo e duplicherebbe l'informazione già data da `aria-expanded`: è quindi marcato `aria-hidden="true"`. Ogni domanda è inoltre racchiusa in un `<h3>`, così che la lista delle FAQ sia percorribile anche navigando per intestazioni.

Quando una risposta è chiusa, oltre a `max-height: 0` le viene applicato `visibility: hidden`, con la transizione ritardata a fine animazione: il testo non resta leggibile agli screen reader mentre `aria-expanded` dichiara il blocco chiuso.

6.2 Contrasto cromatico

L'analisi automatica ha inizialmente rilevato **200 violazioni** del criterio WCAG 2.1 AA sul contrasto (1.4.3), distribuite su tutte e cinque le pagine. Le cause erano tre:

Causa	Rapporto misurato	Richiesto
Oro #B8821A come testo su fondo pergamena (tutte le etichette in maiuscoletto)	2,93:1	4,5:1
Testo avorio semitrasparente su fondo scuro con opacità troppo bassa (piè di pagina a .25)	2,07:1	4,5:1
Testo avorio sul pulsante oro	3,30:1	4,5:1

Gli interventi hanno preservato l'identità visiva invece di appiattire la palette:

- l'oro di marca `--gold` resta invariato per **filetti, bordi, pallini e sfondi**, dove il criterio sul contrasto del testo non si applica;
- per il **testo** sono state introdotte due varianti calibrate sul fondo: `--gold-text: #845C10` sui fondi chiari (minimo 4,84:1) e `--gold-on-dark: #C9942B` sui fondi scuri (minimo 5,93:1);
- le opacità del testo chiaro su fondo scuro sono state portate al valore minimo che soddisfa il criterio (0,62 sull'inchiostro, 0,72 sul verde archivio);
- il pulsante oro ha ora testo color inchiostro (5,35:1), lo stesso trattamento già usato dalle altre etichette su fondo oro.

Al termine, le violazioni rilevate sono **0** su tutte le pagine e su entrambe le larghezze di viewport.

6.3 Struttura, landmark e collegamento di salto

Il contenuto di ogni pagina è racchiuso in `<main id="contenuto">`; con `<nav>` e `<footer>` questo garantisce che nessuna porzione di pagina resti fuori da un landmark, e permette agli utenti di screen reader di saltare direttamente al contenuto.

La prima tabulazione di ogni pagina incontra un **collegamento di salto**, nascosto fuori dallo schermo con `transform: translateY(-120%)` e reso visibile da `.skip-link:focus`. Senza di esso, un utente che naviga da tastiera dovrebbe attraversare l'intero menu su ogni pagina prima di raggiungere il testo.

La gerarchia delle intestazioni è continua su tutte le pagine: un solo `<h1>`, sezioni `<h2>`, blocchi `<h3>`, senza salti di livello.

6.4 Focus, movimento e modulo

- **Focus visibile.** Una regola `:focus-visible` unica applica un contorno oro a ogni elemento attivabile, con una variante più chiara sui fondi scuri. I campi del modulo avevano outline: none e affidavano il segnale di focus al solo colore del bordo: la dichiarazione è stata rimossa.
- **Movimento.** Le animazioni di ingresso e lo scorrimento morbido sono racchiusi in `@media (prefers-reduced-motion: no-preference)`; a chi ha richiesto meno animazioni nelle impostazioni di sistema il sito risponde senza transizioni.
- **Modulo.** Le etichette sono associate ai campi tramite `for/id`. Il campo del messaggio, obbligatorio lato server, porta ora anche l'attributo `required`, così l'errore viene segnalato dal browser prima dell'invio invece di costare un cambio di pagina. Nella scheda dei recapiti alcuni `<label>` erano usati come testo decorativo, senza alcun controllo da etichettare: sono stati sostituiti da `<span>`.

6.5 Validazione W3C

Tutte le pagine, **incluse l'informativa privacy ed entrambe le varianti della pagina di esito** (conferma ed errore), superano la validazione senza errori né avvisi. Le segnalazioni emerse durante lo sviluppo e come sono state risolte:

Segnalazione	Causa	Soluzione
aria-label on div	Attributo ARIA su un <code><div></code> privo di ruolo	Elemento riscritto come <code><button></code> nativo (§6.1)
Skipping heading level	<code><h4></code> sotto sezioni gestite con <code><h2></code>	Alberatura dei titoli ristrutturata su <code><h3></code>
Skipping heading level (piè di pagina)	I titoli di colonna erano <code><h3></code> ; nella pagina di esito, dove il titolo principale è un <code><h1></code> e non esistono sezioni <code><h2></code> , saltavano un livello	Titoli di colonna portati a <code><h2></code>
Stili inline	Attributi <code>style="..."</code> nei blocchi di layout	Regole centralizzate nel solo <code>style.css</code>

6.6 Nessun cookie e nessun terzo

Il sito non imposta cookie — né propri né di terze parti — non usa strumenti di analisi statistica e, una volta ospitati localmente i caratteri tipografici (§3.2), non effettua

alcuna richiesta a domini esterni. Di conseguenza **non è necessario alcun banner di consenso ai cookie**: non c'è nulla da consentire.

È una scelta che vale la pena rendere esplicita, perché il banner è oggi la principale fonte di attrito nella navigazione, e nella grande maggioranza dei siti serve a giustificare trattamenti che il sito potrebbe semplicemente non fare. Qui l'ordine è stato invertito: prima si è eliminato il trattamento, poi è venuto a mancare il motivo del banner.

L'informativa in `privacy.php` documenta comunque la situazione, perché l'assenza di cookie va dichiarata quanto la loro presenza, e descrive l'unico trattamento che l'utente non può evitare: la registrazione degli accessi nei file di log del server web.

7. INSTALLAZIONE E COLLAUDO IN LOCALE

1. Avviare i moduli **Apache** e **MySQL/MariaDB** del proprio ambiente locale (XAMPP, MAMP o WAMP).
2. Copiare la cartella del progetto nella directory servita dal server (per esempio `C:\xampp\htdocs\algora_site` oppure `C:\wamp64\www\algora_site`).
3. Aprire **phpMyAdmin** (<http://localhost/phpmyadmin>) ed eseguire lo script `crea_db.sql`, che crea il database `algora_db` e la tabella `contatti`.
4. Verificare che i parametri in `config-db.php` corrispondano alla propria installazione (i valori predefiniti sono quelli di XAMPP/MAMP: utente `root`, password vuota).
5. Aprire http://localhost/algora_site/index.php.
6. Per collaudare la parte server-side, aprire http://localhost/algora_site/contatti.php, compilare il modulo e inviarlo. Va verificato che compaia la pagina di conferma e che la riga sia presente nella tabella `contatti`.
7. Per collaudare l'area riservata (§5.7), aprire http://localhost/algora_site/archivio.php — o seguire il collegamento “Area riservata” nel piè di pagina — ed entrare con la password dimostrativa `algora2025`. Per sostituirla si rigenera l'impronta da riga di comando e si aggiorna `config-admin.php`:

```
php -r 'echo password_hash("nuova-password", PASSWORD_DEFAULT), "\n";'
```

7.1 Casi di prova eseguiti

Caso	Esito atteso	Esito
Invio corretto, con consenso	Pagina di conferma, riga inserita	Superato
Invio senza consenso al trattamento	Errore specifico, nessun inserimento	Superato
Consenso falsificato (valore diverso da quello atteso)	Errore specifico, nessun inserimento	Superato
Nome con apostrofo e & (Sant'Angelo & C.)	Caratteri corretti a video e nel database	Superato
Email formalmente non valida	Messaggio di errore specifico, nessun inserimento	Superato

Caso	Esito atteso	Esito
Campi obbligatori vuoti	Messaggio di errore specifico, nessun inserimento	Superato
Apertura diretta di <code>invia-contatto.php</code> via URL	Redirect 303 al modulo	Superato
Stringa di SQL injection nel campo nome	Memorizzata come testo, tabella intatta	Superato
Database irraggiungibile	Messaggio generico all'utente, dettaglio nel log, nessun dato tecnico esposto	Superato
Area riservata: password errata	Messaggio generico, nessun accesso	Superato
Area riservata: POST di eliminazione senza sessione valida	Richiesta respinta, riga intatta	Superato
Area riservata: POST privo di token anti-CSRF	Richiesta respinta, riga intatta	Superato
Modifica con email non valida	Errore specifico, valori digitati riproposti nel modulo, nessuna scrittura	Superato
Doppio salvataggio con consenso già presente	<code>data_consenso</code> invariata	Superato
Revoca del consenso in modifica	<code>consenso_privacy</code> a 0 e <code>data_consenso</code> azzerata	Superato
Eliminazione di un numero inesistente	Messaggio specifico, nessuna riga toccata	Superato
Nome contenente <code></code> e apostrofo, mostrato in elenco	Stampato come testo, non interpretato dal browser	Superato

7.2 Verifica del passaggio a mobile-first

La riscrittura del foglio di stile non doveva cambiare il risultato a video. Per dimostrarlo sono state acquisite le schermate a pagina intera delle cinque pagine a 390, 768, 960, 961 e 1440 px prima e dopo l'intervento, e confrontate pixel per pixel:

- a 961 e 1440 px le immagini sono risultate **identiche**, a conferma che la disposizione desktop è invariata;
- a 390, 768 e 960 px l'unica differenza misurata riguarda l'altezza del piè di pagina, cioè esattamente la correzione descritta in §3.4; l'altezza di ogni altro elemento è rimasta invariata.

È stata inoltre verificata l'assenza di scorrimento orizzontale da 320 a 1920 px.

Lo stesso metodo è stato applicato al consolidamento dei nomi di classe descritto in §3.2, che per definizione non doveva cambiare nulla a video: ventotto schermate a pagina intera (le sette pagine per quattro larghezze) sono state confrontate pixel per pixel prima e dopo l'intervento e sono risultate **tutte identiche**, insieme alle due schermate di stato — FAQ aperta e menu mobile aperto — e alle tre larghezze estreme. I primi confronti avevano invece segnalato differenze reali, tutte dovute a

regole che avevano perso il confronto di specificità: sono state la guida per correggere il consolidamento prima di considerarlo concluso.

8. PUBBLICAZIONE ONLINE

Il sito è pensato per essere pubblicato sul dominio `www.algorastudio.it`. Su hosting condiviso la procedura è la seguente:

1. Caricare i file del progetto nella cartella pubblica del dominio (tipicamente `public_html` o `htdocs`).
 2. Creare il database dal pannello dell'hosting ed eseguirvi `crea_db.sql`.
 3. Aggiornare `config-db.php` con host, nome del database, utente e password forniti dal gestore del dominio, assegnando all'utente i soli privilegi necessari (INSERT e SELECT sulla tabella contatti).
 4. Verificare che `config-db.php` non sia raggiungibile dall'esterno e che non venga incluso nel versionamento.
-

9. LIMITI NOTI E SVILUPPI FUTURI

Per completezza si segnalano gli aspetti ancora aperti:

- **Contenuti multimediali.** Il sito è tuttora interamente testuale. L'impianto per accogliere le schermate di Foliarium è predisposto nella scheda prodotto — struttura `<figure>`, didascalie, testi alternativi, dimensioni dichiarate per evitare lo slittamento del layout — ma resta disattivato in attesa delle immagini. Le istruzioni sono in `img/LEGGIMI.md`.
- **Dati del titolare nell'informativa.** L'informativa privacy è completa nella struttura, ma i riferimenti anagrafici del titolare (ragione sociale, partita IVA, sede), il periodo di conservazione e i fornitori nominati responsabili esterni sono segnaposto, resi graficamente evidenti nella pagina. Vanno compilati prima della pubblicazione: un'informativa pubblicata a metà è peggio di nessuna informativa. Lo stesso segnaposto della partita IVA compare nel piè di pagina.
- **Protezione del modulo pubblico.** Il token anti-CSRF descritto in §5.7 protegge l'area riservata, ma non è stato esteso al modulo di contatto, che resta privo anche di una misura anti-spam. Per un uso in produzione andrebbero aggiunti entrambi. Per la seconda è preferibile una tecnica passiva — campo `esca` più controllo sul tempo di compilazione — rispetto a un CAPTCHA: quelli visuali sono un ostacolo di accessibilità, e i servizi di terze parti reintrodurrebbero in pagina la dipendenza esterna eliminata in §3.2.
- **Autenticazione dell'area riservata.** L'accesso a `archivio.php` poggia su un'unica password condivisa: basta a proteggere un pannello dimostrativo, non un archivio in produzione, che richiederebbe utenze nominali, HTTPS obbligatorio, limitazione dei tentativi di accesso e registro delle operazioni eseguite. Manca inoltre una cancellazione programmata al termine del periodo di conservazione: oggi la cancellazione è un gesto manuale, per quanto ora possibile senza aprire phpMyAdmin.